

Gaussowski Naiwny klasyfikator Bayesa

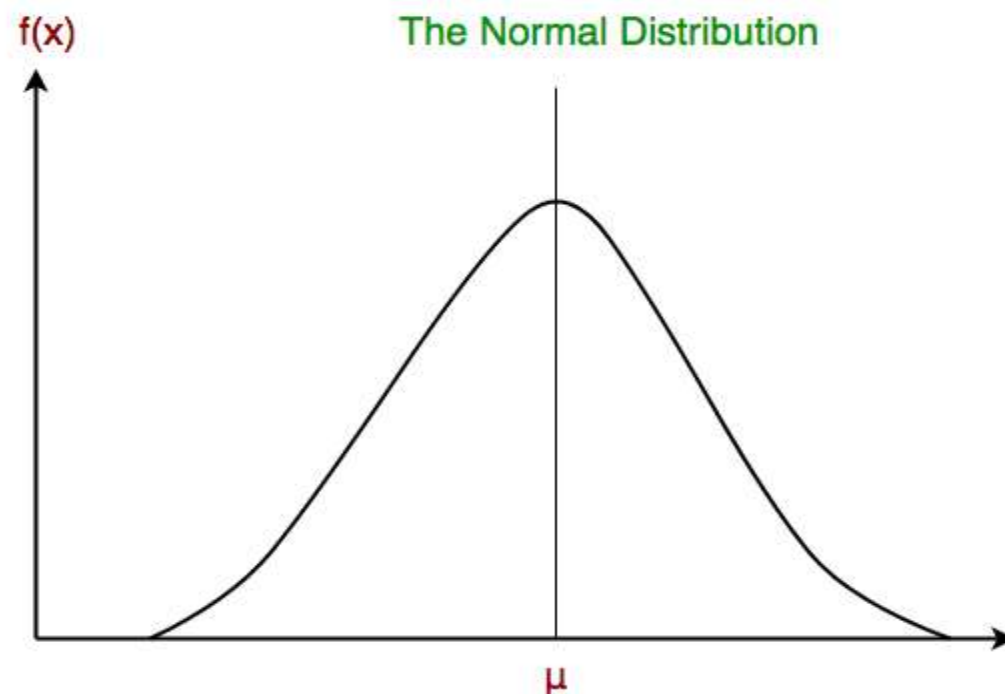
W Gaussowskim Naiwnym Bayesie przyjmuje się, że **ciągłe wartości związane z każdą cechą (a posteriori), mają być rozłożone zgodnie z rozkładem Gaussa.**

Rozkład Gaussa jest również nazywany **rozkładem normalnym**. Po wykreśleniu daje krzywą w kształcie dzwonu, która jest symetryczna względem średniej wartości cech, jak pokazano poniżej:

Loadings:

1. Otwórz Anaconda Navigator
2. włącz Jupyter Notebook
3. Znajdź html file : [Gaussian Naive Bayes.html](#)
It is a pattern for Your own work.

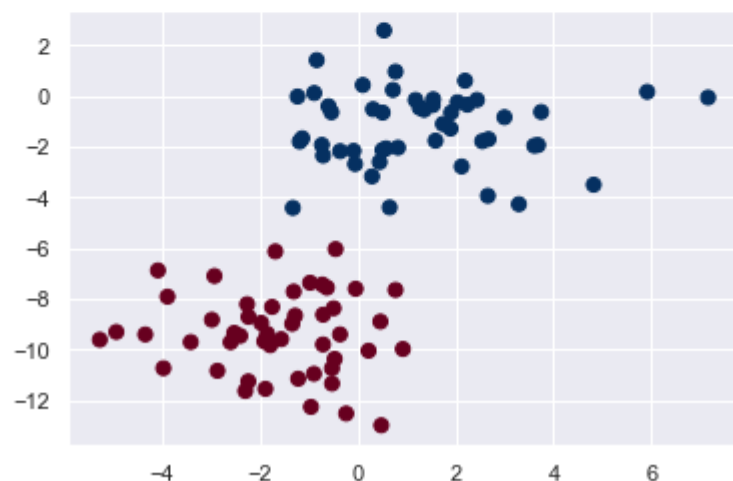
Założyliśmy a posteriori, że np. rozmieszczenie sinic i okrzemek w Zalewie Wiślanym jest gaussowskie



```
In [1]: #Gaussian Naive Bayes
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
```

```
In [2]: #Gaussian Naive Bayes
#Perhaps the easiest naive Bayes classifier to understand is Gaussian naive Bayes.
#In this classifier, the assumption is that data from each label is drawn from a simple Gaussian distribution.
#Imagine that you have the following data:

from sklearn.datasets import make_blobs
X, y = make_blobs(100, 2, centers=2, random_state=2, cluster_std=1.5)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='RdBu');
```



```
In [3]: # One extremely fast way to create a simple model is to assume that the data is described by
# a Gaussian distribution with no covariance between dimensions.
# This model can be fit by simply finding the mean and standard deviation of the points within each label,
# which is all you need to define such a distribution.
# The result of this naive Gaussian assumption is shown in the following figure:
```

```

In [5]: from sklearn.datasets import make_blobs
X, y = make_blobs(100, 2, centers=2, random_state=2, cluster_std=1.5)

fig, ax = plt.subplots()

ax.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='RdBu')
ax.set_title('Naive Bayes Model', size=14)

xlim = (-8, 8)
ylim = (-15, 5)

xg = np.linspace(xlim[0], xlim[1], 60)
yg = np.linspace(ylim[0], ylim[1], 40)
xx, yy = np.meshgrid(xg, yg)
Xgrid = np.vstack([xx.ravel(), yy.ravel()]).T

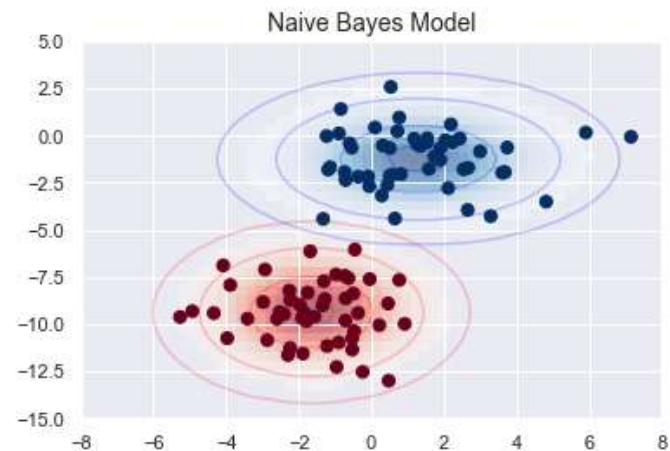
for label, color in enumerate(['red', 'blue']):
    mask = (y == label)
    mu, std = X[mask].mean(0), X[mask].std(0)
    P = np.exp(-0.5 * (Xgrid - mu) ** 2 / std ** 2).prod(1)
    Pm = np.ma.masked_array(P, P < 0.03)
    ax.pcolorfast(xg, yg, Pm.reshape(xx.shape), alpha=0.5,
                  cmap=color.title() + 's')
    ax.contour(xx, yy, P.reshape(xx.shape),
               levels=[0.01, 0.1, 0.5, 0.9],
               colors=color, alpha=0.2)

ax.set(xlim=xlim, ylim=ylim)

fig.savefig

```

Out[5]: <bound method Figure.savefig of <Figure size 432x288 with 1 Axes>>

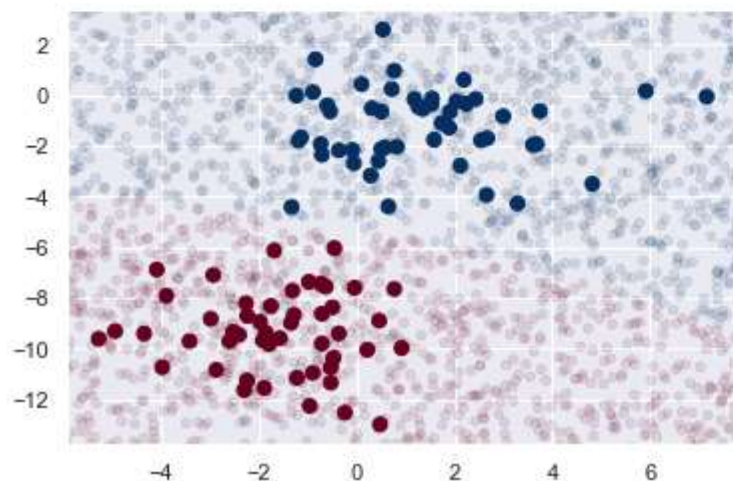


```
In [7]: # The ellipses here represent the Gaussian generative model for each label,  
# with larger probability toward the center of the ellipses.  
# With this generative model in place for each class,  
# we have a simple recipe to compute the likelihood  $P(\text{features} \mid L1)$  for any data point,  
# and thus we can quickly compute the posterior ratio and determine which label is the most probable for a given point.  
  
#This procedure is implemented in Scikit-Learn's sklearn.naive_bayes.GaussianNB estimator:
```

```
In [8]: from sklearn.naive_bayes import GaussianNB  
model = GaussianNB()  
model.fit(X, y);
```

```
In [9]: # Now Let's generate some new data and predict the label:  
rng = np.random.RandomState(0)  
Xnew = [-6, -14] + [14, 18] * rng.rand(2000, 2)  
ynew = model.predict(Xnew)
```

```
In [10]: # Now we can plot this new data to get an idea of where the decision boundary is:  
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='RdBu')  
lim = plt.axis()  
plt.scatter(Xnew[:, 0], Xnew[:, 1], c=ynew, s=20, cmap='RdBu', alpha=0.1)  
plt.axis(lim);
```



```
# We see a slightly curved boundary in the classifications—in general, the boundary in Gaussian naive Bayes is quadratic.  
# A nice piece of this Bayesian formalism is that it naturally allows for probabilistic classification,  
# which we can compute using the predict_proba method:
```

Funkcja **predict_proba()** w klasyfikacji wektorów nośnych (SVC) pakietu Scikit-learn to narzędzie, które pozwala użytkownikom uzyskiwać oszacowania prawdopodobieństwa dla przewidywanych klas.

Rola **predict_proba()**

Funkcja **predict_proba()** została zaprojektowana w celu uzyskania oszacowań prawdopodobieństwa dla każdej etykiety klasy w zadaniu klasyfikacji. Jest to szczególnie przydatne w aplikacjach, w których zrozumienie pewności prognozy jest równie ważne, jak sama prognoza.

Mechanizm wewnętrzny: skalowanie Platta

Funkcja **predict_proba()** w SVC wykorzystuje metodę [skalowania Platta](#) do konwersji wartości decyzyjnych z SVM na prawdopodobieństwa. Skalowanie Platta to etap przetwarzania końcowego, który stosuje model regresji logistycznej do wartości decyzyjnych, czyli odległości próbek od hiperpłaszczyzny.

Po wytrenowaniu, model regresji logistycznej jest używany do konwersji wyników funkcji decyzyjnych na prawdopodobieństwa. Odbywa się to dla każdej klasy, co skutkuje rozkładem prawdopodobieństwa dla wszystkich klas.

Etapy skalowania Platta

- Szkolenie SVM** : Początkowo SVM jest szkolony w celu znalezienia optymalnej hiperpłaszczyzny, która oddziela klasy w przestrzeni cech.
- Funkcja decyzyjna**: Funkcja decyzyjna oblicza odległość każdej próbki od hiperpłaszczyzny ze znakiem. Odległość ta jest wykorzystywana jako wartość wejściowa do skalowania Platta.
- Regresja logistyczna**: skalowanie Platta dopasowuje model regresji logistycznej do wartości decyzyjnych, optymalizując parametry A i B w taki sposób, aby prawdopodobieństwo:

$$P(i = 1 | X) = \frac{1}{1 + \exp(A \cdot f(X) + B)}$$

gdzie $f(X)$ jest wartością decyzyjną dla próbki X

Cechą modelu bayesowskiego jest to, że w naturalny sposób pozwala on na klasyfikację probabilistyczną, którą możemy obliczyć za pomocą metody **predict_proba**:

```
In [11]: # We see a slightly curved boundary in the classifications—in general, the boundary in Gaussian naive Bayes is quadratic.
# A nice piece of this Bayesian formalism is that it naturally allows for probabilistic classification,
# which we can compute using the predict_proba method:
yprob = model.predict_proba(Xnew)
yprob[-8:].round(2)
```

```
Out[11]: array([[0.89, 0.11],
               [1.  , 0.  ],
               [1.  , 0.  ],
               [1.  , 0.  ],
               [1.  , 0.  ],
               [1.  , 0.  ],
               [0.  , 1.  ],
               [0.15, 0.85]])
```

W kolumnach podano prawdopodobieństwa a posteriori odpowiednio pierwszej i drugiej etykiety.

```
In [ ]: # The columns give the posterior probabilities of the first and second label, respectively.
# If you are looking for estimates of uncertainty in your classification,
# Bayesian approaches like this can be a useful approach.

# Of course, the final classification will only be as good as the model assumptions that lead to it,
# which is why Gaussian naive Bayes often does not produce very good results.
# Still, in many cases—especially as the number of features becomes large—this assumption is not detrimental enough
# to prevent Gaussian naive Bayes from being a useful method.
```